

Programming and Databases

Joern Ploennigs

Database types

MIDJOURNEY: DATABASE TREE, REF. GUSTAV KLIMT

RECAP: LECTURE HALL QUESTION

What are inheritance, generalization, encapsulation, and polymorphism?



Midjourney: Object oriented man

RECAP: OBJECT PROPERTIES

Inheritance	Generalization	Polymorphism	Encapsulation
Attributes and methods of parent classes are inherited by child classes. This helps avoid redundancies and errors.	Commonalities are implemented in generalized parent classes	Child classes can override methods and thus redefine them.	Encapsulation of data and methods in objects is a protective mechanism to limit faulty changes.

RECAP: IN-CLASS QUESTION

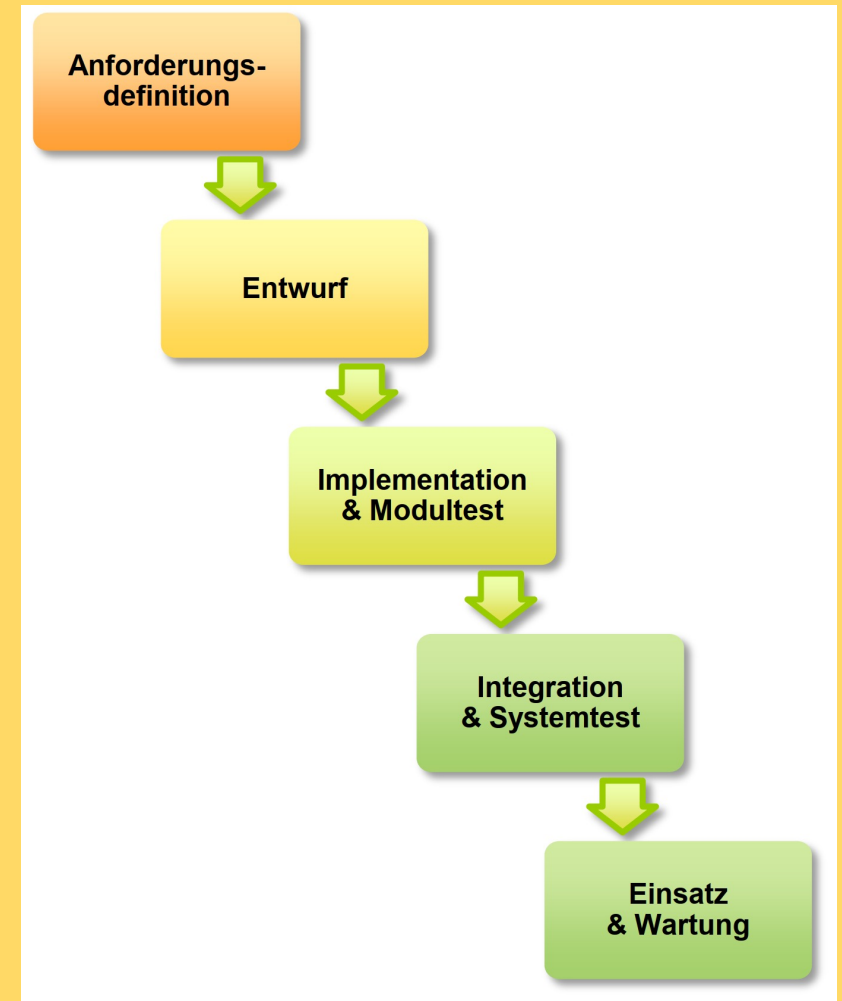
What is the Waterfall Method?



Midjourney: Waterfall

LINEAR METHOD - WATERFALL METHOD

- Traditional model that is still often required in the procurement of large systems
- Development is divided into several sequential steps, and each step must be completed before the next
- User involvement only in the requirements definition
- Each activity is documented — well suited for tenders (after the requirements definition or the design, ISO 9000)



RECAP: IN-CLASS QUESTION

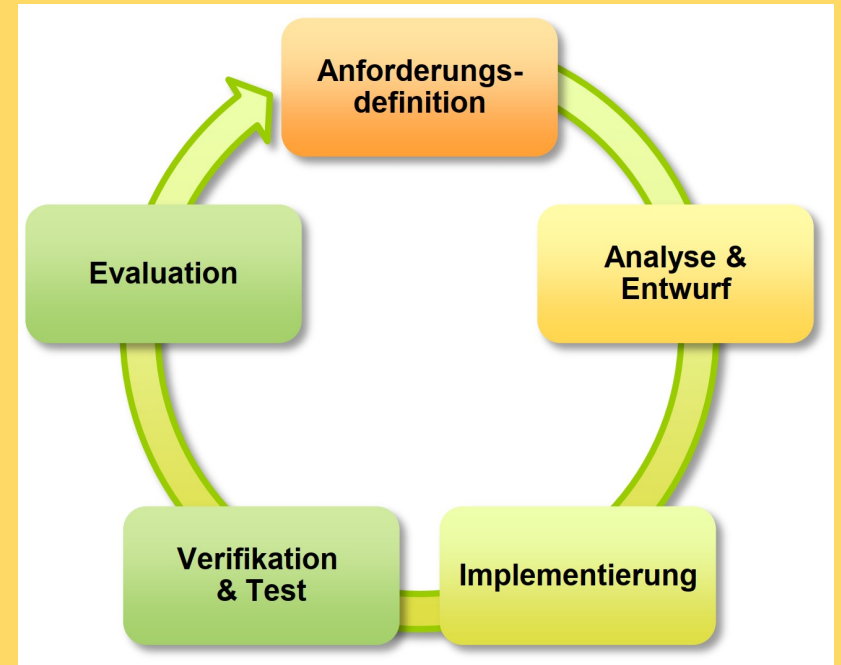
What is the Agile method?



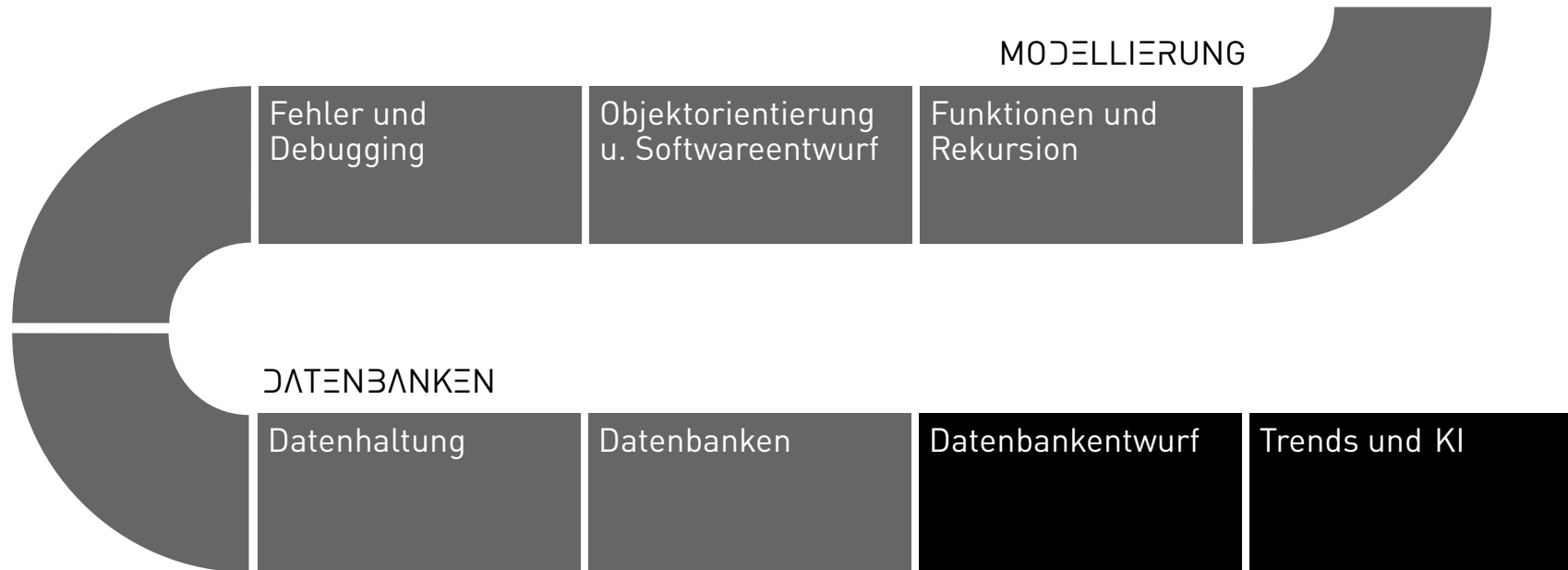
Midjourney: Sprinter and Waterfall

AGILE METHOD

- A modern method to adapt to constantly changing requirements
- The goal of incremental development of the solution is to keep the effort and complexity of individual steps in check
- Starts with a simple and extensible implementation (MVP – Minimum Viable Product)
- Incremental expansion and improvement of the product with regular releases (usually every 3 months)
- Design flaws in early iterations can lead to a complete redesign



PROCESS



DATABASE DEFINITION

Definition: Database

A database (DB) refers to the logically related data that is managed by a *DBMS (Database Management System)*.

Definition: Database System

A database and the database management system together are referred to as a *Database System*.

In addition to the pure 'user data', a database also includes the objects created for the DBMS to manage (for example, indexes and log files).

WHY DO WE PUT MONEY IN THE BANK?

- Centralized and long-term storage
- Protection against losses
- Efficiency through specialized service offerings (standing orders, portfolio, ...)
- Keeping an overview
- Networking with the global financial network



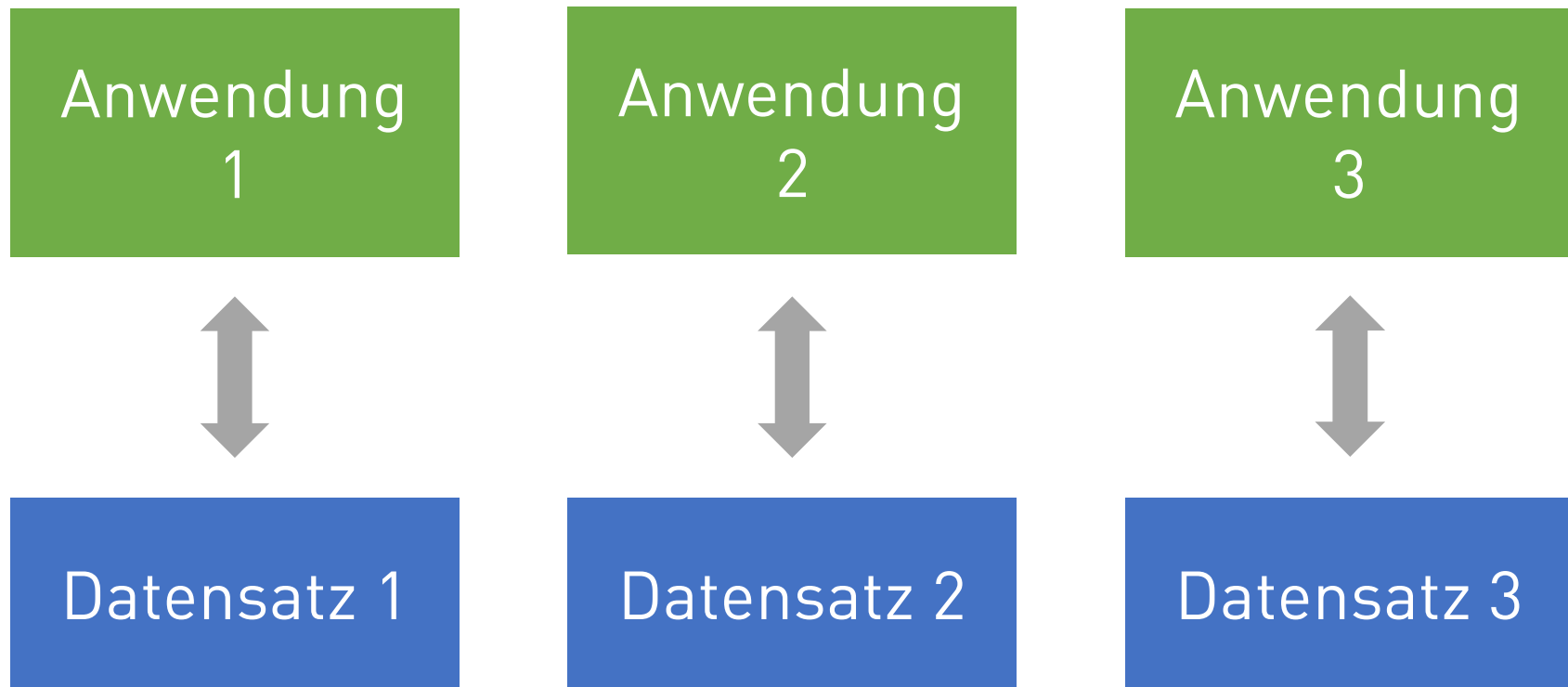
Midjourney: thief breaking into a bank safe

WHY DO WE PUT DATA INTO THE 'BANK'?

- *Efficiently* store and load data across different clients (web servers, devices, etc.)
- *Management* of very (very) large data volumes (scalability)
- *Organization* of the data into predefined data structures (normalization)
- *Long-term* storage of the data (persistence)
- *Fast* search of the data through indexing
- *Safeguarded* processes for changing data (transactions)
- *Auditable* changes to data via transaction logs
- *Automatic* data analysis (OLAP)

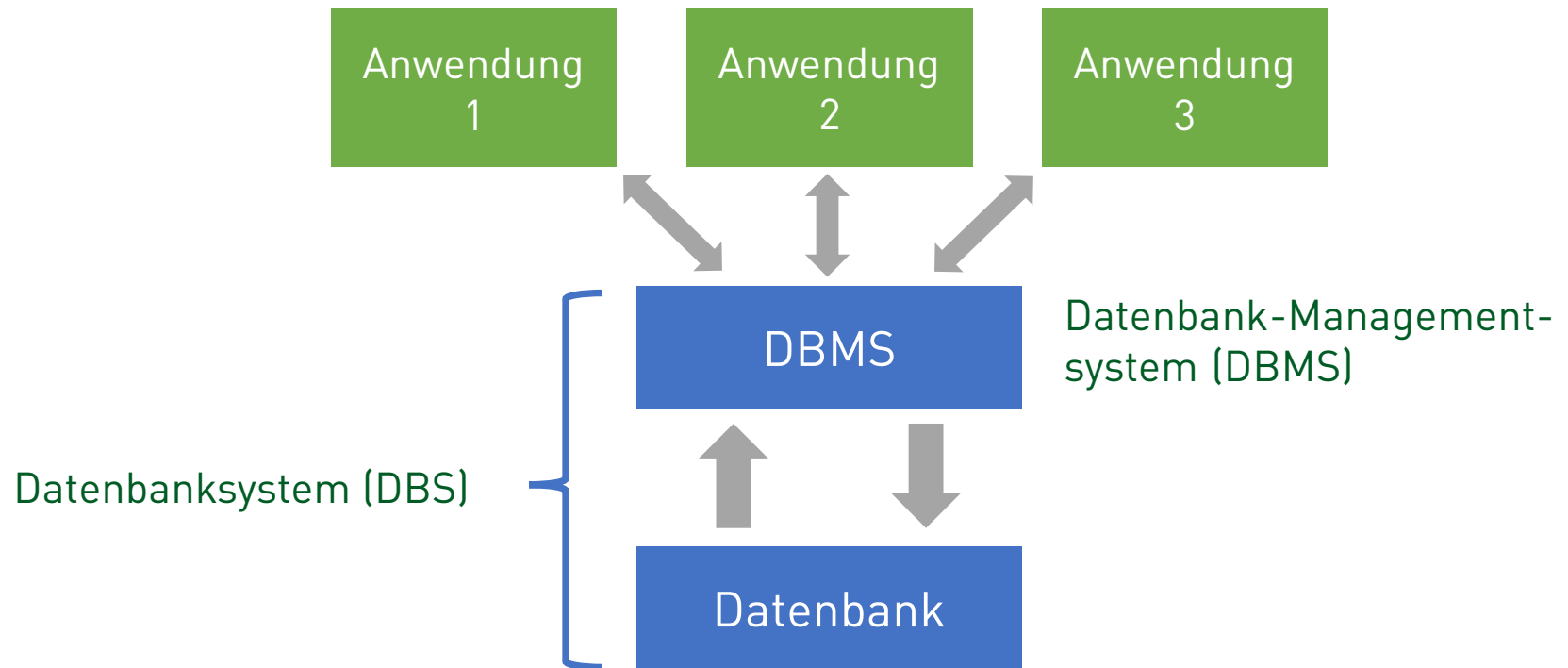
CORE CONCEPTS: FILES

- Structure: Each application structures the data according to the data types present therein (format, structure, ...).
- File system: Each application stores the data according to its requirements (access, extension, location, ...).



CORE CONCEPTS OF DATABASES

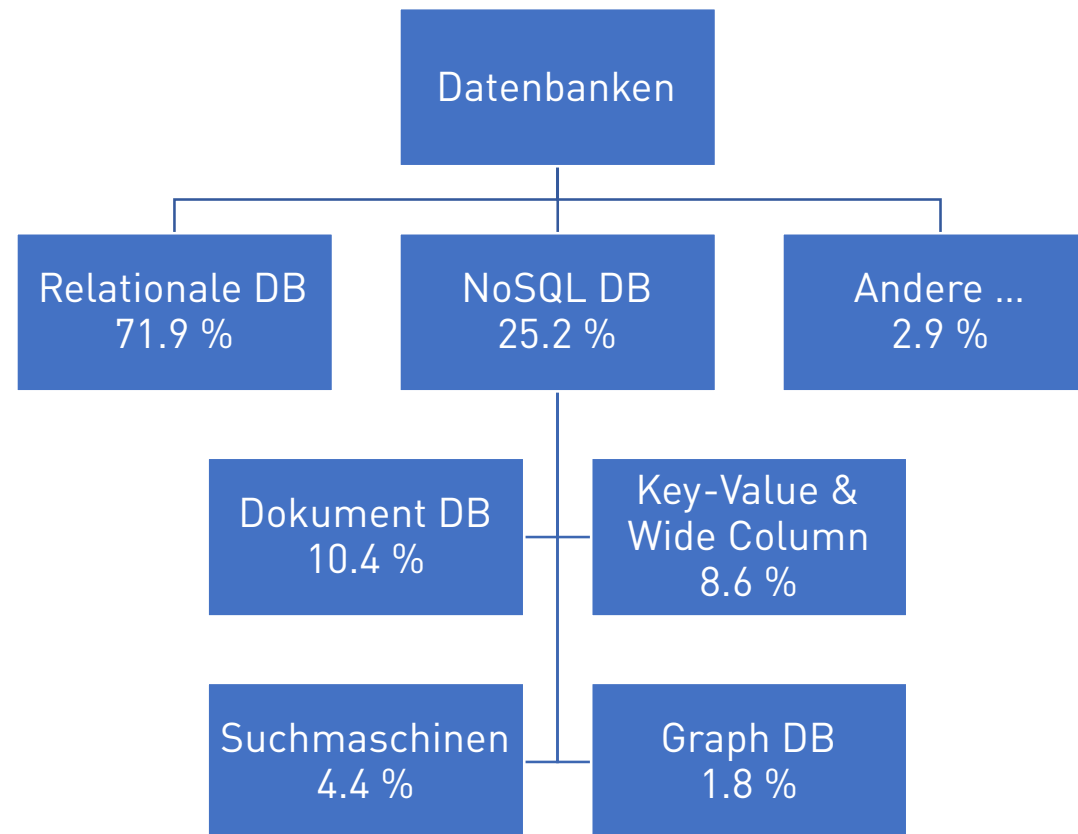
- Structure: All applications use the same structure that is modeled in the database
- File system: All applications access the same data. Accesses are synchronized and logged via transactions.



THE CODD RULES (Codd, 1985, 1990)

- *Integration*: unified, non-redundant data management
- *Operations*: Store, Search, Modify
- *Catalog*: Access to database descriptions in the Data Dictionary
- *User Views*: Each user sees the data they are allowed to see, in the way they want to view it
- *Integrity*: Correctness of the database contents
- *Data Security*: Prevention of unauthorized access; only authorized users
- *Transactions*: Multiple DB operations as a single unit (all or nothing)
- *Synchronization*: Coordinating parallel transactions
- *Backup and Recovery*: Recovery of data after system failures

DATABASE TYPES - OVERVIEW



RELATIONAL DATABASES

RDBMS have been in use since the early 1980s and are based on the relational (= tabular) data model

- The schema of a table (= relation schema) is defined by the table name and a fixed set of attributes (= columns) with corresponding data types
- Because data are organized in tables, they are highly structured with a structure defined by the table (normalization)
- The standard language for creating/modifying/deleting is *SQL*
- Popular systems: Oracle, MySQL, Microsoft SQL Server, PostgreSQL, IBM Db2

NoSQL DATABASES

NoSQL database management systems are databases that do not use a relational (= table-oriented) data model and therefore typically do not support SQL.

- They have been increasingly widespread since around 2009.
- Popular systems: MongoDB, CouchDB, Cassandra, Redis, Neo4j, Amazon DynamoDB, HBase, OrientDB
- *Main reasons:*
 - High scalability requirements
 - Fault tolerance of modern web applications
 - Big data scenarios
 - Data is often only semi-structured (they do not fit neatly into a schema)

DOCUMENT-ORIENTED DATABASES

Document Stores are characterized by a *schema-free* organization of the data:

- Documents have no uniform structure
- The data types of values in individual fields can vary from document to document
- Fields can contain more than one value (arrays)
- Documents can have a nested structure
- To represent the “documents,” JSON is most commonly used
- Popular systems: MongoDB, Amazon DynamoDB, Databricks, Azure Cosmos DB, Couchbase

KEY-VALUE DATABASES

- Key-Value stores are arguably the simplest form of database management systems
- They can store only key–value pairs, and retrieve the values by key
- They thus resemble Python's *dict*
- This simplicity makes them attractive for:
 - resource-constrained systems such as embedded PCs
 - development of web interfaces
- Popular systems: Redis, Amazon DynamoDB, Azure Cosmos DB, Memcached, Hazelcast

SEARCH ENGINE DATABASES

Search engines are NoSQL DBMS specialized in searching data content such as text:

- Features
 - Support for complex search terms
 - Full-text search
 - Stemming (reduction to the word stem)
 - Result ranking
 - Grouping of search results
 - Distributed search for high scalability
- Popular systems: Elasticsearch, Splunk, Solr, OpenSearch, MarkLogic

GRAPH DATABASES

Graph DBMS represent data as *nodes (Nodes)* and *relationships (Edges)* to each other

- They enable, in particular, the *modeling of connections*
- Ideal for network analysis, social networks, and recommendation systems
- Popular systems: Neo4j, Microsoft Azure Cosmos DB, Virtuoso, IBM KITT

DATABASE TYPES IN THE LECTURE

- Focus on: *Relational DB* (71.9%)
 - Easy to use
 - Most widely used
 - Solid foundation
- Also: Key-Value DB:
 - Simple concepts
 - Practical exercises

DATABASES IN PYTHON

- Access via the appropriate library for each database
- Oracle, Redis, MongoDB, etc.

```
from replit import db

# Save data
db["entry"] = 5

# Retrieve data
value = db["entry"]
print(value)  # Output: 5
```


Questions?

